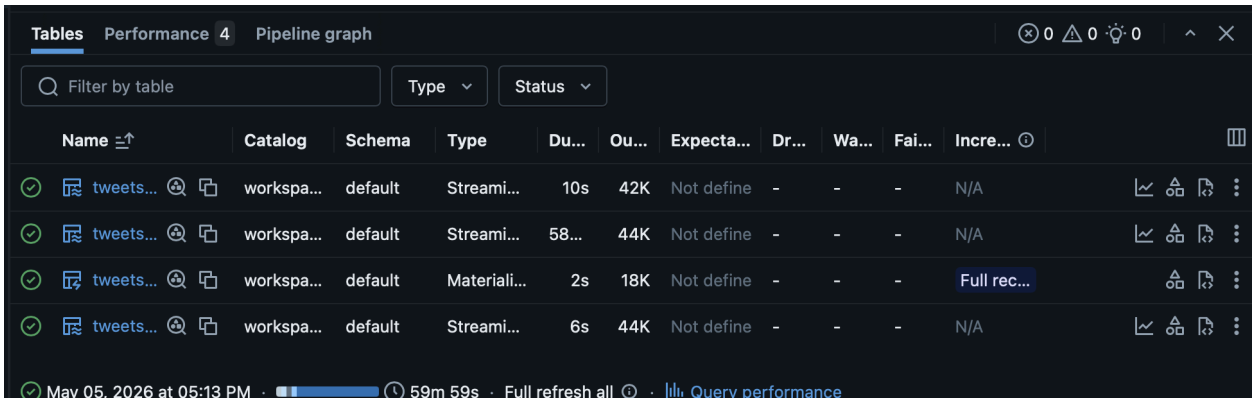
Tweet Sentiment Pipeline Performance Optimization Analysis

My final pipeline completed a full refresh successfully in about 59 minutes and 59 seconds. The bronze layer produced about 42K rows, the silver layer produced about 44K rows after mention explosion, the gold layer produced about 44K rows with model predictions, and the aggregation layer produced about 18K rows. Bronze and silver completed quickly, so the main performance bottleneck was the gold layer where the MLflow/Hugging Face sentiment model was applied.

Serialization / UDF Overhead:

The gold layer was the slowest part of the pipeline because each Spark worker had to load and run a Hugging Face Transformers model through MLflow. This creates Python serialization overhead, model artifact download overhead, tokenizer/model loading time, and Torch inference overhead. During development, I also saw memory-related failures when the model exceeded the function memory limit. To reduce this risk, I used a smaller DistilBERT sentiment model and pinned the gold layer to the correct Unity Catalog model version. For a production pipeline, I would continue using a smaller model, and avoid unnecessary model reloads.



Shuffle:

The application layer groups records by mention to calculate positive, negative, and total mention counts. This groupBy operation requires a shuffle because all rows for the same mentioned user must be moved to the same partition. The dataset size was manageable, so the aggregation completed quickly, but this could become expensive on a larger tweet dataset. To reduce shuffle cost, I filtered out NULL mentions before aggregation and only selected the columns needed for the dashboard. For larger data, I would tune spark.sql.shuffle.partitions and consider repartitioning by mention before the aggregation.

Storage and Checkpoint State:

Because the pipeline uses streaming tables and checkpoint state, failed runs or frequent code changes can leave checkpoint state inconsistent with the current pipeline code. This caused some runs to spin for a long time during development. Resetting the checkpoint volume and dropping the target tables between major code changes made the pipeline behavior more

predictable. I also tested the pipeline first on the smaller test tweet dataset before running the full dataset. For production, I would avoid unnecessary full refreshes after a successful run and rely on incremental processing.

Skew:

The mention aggregation may be affected by skew because some users or celebrities receive many more mentions than others. This means a few mention values can have much larger groups than the rest. In the dashboard results, the top mentioned accounts were much more frequent than most others. If the dataset were larger, I would inspect the distribution of mention counts and use salting or pre-aggregation if a few keys caused slow tasks.

Spill Risk:

The current dataset did not appear large enough to make the aggregation layer a major bottleneck, but larger input volumes could cause shuffle spill during the groupBy operation. I would monitor Spark UI metrics such as shuffle read, shuffle write, memory spill, and disk spill. If spill increased, I would tune partition counts, reduce columns before the shuffle, and consider caching or optimizing intermediate Delta tables.

Recommendations:

The most important optimization is to control the gold-layer model inference cost. The pipeline should use a verified, smaller Unity Catalog model version and avoid re-registering or accidentally loading the wrong model version. During development, the safest workflow is to test on the small tweet dataset, reset checkpoints between major code changes, and only run the full dataset once the end-to-end test succeeds.